

[C++_QT_ 3일차 알고리즘 보고서]

이름: 김도훈

학번: 2026406041

학과: 로봇학부

<목차>

1. 알고리즘이란?
2. 최단 경로 알고리즘
 - 2-1 다익스트라(Dijkstra) 알고리즘
 - 2-2 a* 알고리즘
 - 2-3 C++에서 구현하기
 - 2-3-(1) 다익스트라 C++에서 구현하기
 - 2-3-(2) A*start C++에서 구현하기

1. 알고리즘이란?

알고리즘이란 컴퓨터가 따라 할 수 있도록 문제를 해결하는 절차나 방법을 자세히 설명하는 방법이다. 자세히 설명하면 컴퓨터를 활용한 문제 해결 과정에서 주어진 문제를 해결하는 일련의 방법 또는 절차이다.

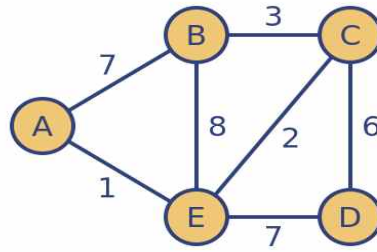
2. 최단 경로 알고리즘

최단 경로 알고리즘은 그래프에서 두 노드 사이의 비용을 가장 적게 만드는 경로를 찾는 방법이다. 여기서 노드는 공간이나 도표에서 위치를 나타내는 하나의 점을 뜻하는 정점이다. 그리고 비용은 가중치이다. 예를 A 정점에서 B 정점으로 이동할 때 걸리는 시간, 거리, 위험도 등으로 말할 수 있다. 우리가 알아야 할 최단 경로 알고리즘은 다익스트라, A* 알고리즘이 있다.

2-1 다익스트라(Dijkstra) 알고리즘

다익스트라 알고리즘은 노드 간에 연결된 간선들이 가중치를 가지고 있어 이 가중치를 이용하여 그래프의 최단 경로를 찾는 알고리즘이다. 일단 다익스트라 알고리즘의 로직은 다음과 같다.

1. 초기화 - 시작 노드 거리(가중치)는 0으로 두고 시작 노드를 제외한 노드는 무한으로 설정을 해둔다.
2. 노드 이동 - 시작 노드와 간선으로 연결된 노드(방문 하지 않은 노드) 중에서 거리가 가장 짧은 최단 거리(최소 비용)를 가진 노드를 선택한다.
3. 업데이트 - 이동한 노드에서도 계속해서 최소 비용을 계산해서 노드를 이동하여 시작점에서 모든 노드까지의 최단 거리가 정해질 때 까지 반복한다..
4. 이동 - 최단 비용이 가장 작은 노드로 이동을 한다.



노드가 5개인 그래프가 있다. 이 그래프에 다익스트라 알고리즘을 사용해 각 노드들의 최단 경로를 구해보겠다. 시작점 노드는 노드 A이다. 최초 상태에서 A가 시작점이기 때문에 비용은 0이다. A를 제외한 다른 정점들은 무한으로 초기화를 해둔다. 각 노드에 최송 비용을 찾는 것을 시작하지 않았다는 것을 표현하기 위해 무한대로 표현한다.

A	B	C	D	E
0	무한	무한	무한	무한

그리고 노드 A와 연결된 모든 노드들의 비용을 테이블에 표시한다. A와 연결된 노드는 노드 B와 노드 E 이다. 비용은 각각 7과 1이다.

A	B	C	D	E
0	7	무한	무한	1

노드 B와 노드 E 중 가중치가 짧은 것이 노드 E이기 때문에 노드 E로 이동한다. 그리고 다시 연결된 노드를 찾고 이 과정을 반복한다.

A	B	C	D	E
0	7	3	8	1

이 과정에서 A -> B로 가는 것이 7이고 A -> E -> B로 가는 것이 9이다. A -> B로 가는 것이 더 최소 비용이기 때문에 계속해서 7로 유지한다. 이렇게 경로가 여러 가지일 때는 최소 비용인 경로를 선택한다. 그리고 모든 노드들에 접근했을 때 시작 노드(노드A)에서 노드 B까지의 비용이 처음에는 7이었지만 A -> E -> C -> B로 가는 방법이 비용이 6으로 더 적게 나오기 때문에 비용을 노드 B의 비용을 6으로 변경해준다.

A	B	C	D	E
0	6	3	8	1

모든 정점에 접근했기 때문에 반복을 멈춘다. 이때 테이블이 시작점 노드 A로 부터 각 노드에 접근할 수 있는 최소 비용이다. 이것이 다익스트라의 핵심 로직이다.

2-2 a* 알고리즘

$$F(n) = G(n) + H(n)$$

a* 알고리즘은 휴리스틱이라고 불리는 위의 함수의 값이 가장 작은 방향으로 경로를 탐색해 최단 경로를 찾는 알고리즘이다. 기본적으로 다익스트라의 기반을 두고 있다. 다익스트라 알고리즘은 주변에 있는 노드들에 모두 접근해 비용이 최소한 노드를 찾아 그 노드로 이동하여 최단 경로를 탐색하지만 a* 알고리즘은 휴리스틱이라는 다음 방문할 노드의 예상 비용을 계산해주는 함수의 값을 고려하여 최단 경로를 탐색하는 것이다. 이것이 a* 알고리즘과 다익스트라 알고리즘의 가장 큰 차이이다. 다음 방문할 노드의 예상 비용을 계산해준다는 것은 쉽게 말해 시작점에서 도착점이 어디 방향에 위치해 있는지에 대한 값이라고 생각하면 이해하기 쉽다.

$G(n)$ = 시작 노드에서 현재 노드까지의 거리(지금까지 온 거리)

이 값을 구할 땐 만약 코드에서 사용할 맵이 2차원 정사각형 격자 맵이라면 이동한 가로, 세로의 길이가 값이 되는 것이다.

$H(n)$ = 현재 노트에서 목표 지점 노드까지의 거리(목표 지점까지 이동 해야하는 남은 거리)

이 값을 구할 땐 맨해튼 거리 방식을 사용한다. 맨해튼 거리는 격자 형식의 2차원 평면에서 두 점 사이의 거리를 구할 때 대각선이 아닌 가로, 세로의 이동 방향의 합으로 구하는 방식이다.

$F(n)$ = 위의 $G(n)$ 과 $H(n)$ 을 더한 값(비용), 이 값이 작은 곳을 노드를 이동(탐색 방향)

다익스트라 알고리즘은 주변에 있는 모든 노드들에 접근해 비용이 가장 적은 노드를 선택해 최단 경로를 탐색했다면 A* 알고리즘은 휴리스틱 함수의 비용이 가장 적은 노드 쪽으로 탐색을 해 시작 노드에서 도착 노드까지 다익스트라 알고리즘보다 더 적은 노드를 탐색하고 빠른 시간에 최단 경로를 찾을 수 있다는 장점을 가지고 있다.

2-3 C++에서 구현하기

2-3-(1) 다익스트라 알고리즘 C++에서 구현하기

다익스트라 알고리즘을 C++에서 구현해보자.

본 C++ 다익스트라 알고리즘 프로그램은 위에서 다익스트라 알고리즘을 설명할 때 사용한 그래프를 실제로 코드로 구현하여 다익스트라 알고리즘을 사용해 그 그래프에 있는 노드들의 최소 비용을 찾는 프로그램이다. 5개의 노드 A, B, C, D, E 노드를 코드에서는 0, 1, 2, 3, 4로 표현하고 인접 리스트를 사용하여 그래프를 구현하였다. 그리고 각 노드까지의 최단 거리와 방문 여부를 관리하면서 최단 경로 비용을 계산하였다.

```
graph
vector<pair<int, int>> graph[5];
```

5개의 노드 사이의 연결 관계와 간선의 가중치를 저장하기 위해 사용하였다. pair의 첫 번째 값에는 연결된 노드의 번호를 저장하고 두 번째 값에는 해당 노드까지 이동하는 데 필요한 가중치를 저장한다. 예를 들어 graph[0].push_back({1, 7});은 0번 노드와 1번 노드가 연결되어 있고 이동 비용이 7이라는 것을 의미한다.

인접 리스트를 사용하여 현재 노드와 연결된 노드만 확인할 수 있도록 그래프를 구현하였다.

```
dist
vector<int> dist(5, INF);
```

시작 노드에서 각 노드까지의 현재 최단 거리를 저장하기 위해 사용하였다. 알고리즘 시작 시에는 아직 각 노드까지의 경로를 알 수 없기 때문에 모든 값을 INF로 초기화하고 시작 노드의 거리는 0으로 설정하였다. 참고로 INF 변수는 2147483647로 선언했고 그 이유는 무한대를 표현하기 위해 int 자료형의 최댓값을 사용했다.

```
dist[start] = 0;
```

탐색 과정에서 더 짧은 경로를 발견하면 해당 노드의 dist 값을 갱신한다.

```
if(dist[u] + weight < dist[v]){
    dist[v] = dist[u] + weight;
}
```

```
vector<bool> visited(5, false);
```

각 노드의 방문 여부를 확인하기 위해 사용하였다. 처음에는 모든 노드를 방문하지 않았으므로 false로 초기화한다. 다익스트라 알고리즘에서는 방문하지 않은 노드 중 현재 거리가 가장 짧은 노드를 선택하고 선택된 노드는 다음과 같이 방문 처리한다.

```
visited[u] = true;
```

이를 통해 이미 최단 거리가 확정된 노드가 다시 선택되는 것을 방지하였다.

```
vector<int> Dijkstra(int start);
```

다익스트라 알고리즘을 구현한 함수이다. 최소 비용을 가지고 있는 dist 배열을 반환해야하기 때문에 함수의 반환 자료형을 vector<int>로 설정해두었다. 또한 사용자가 선택한 시작 노드를 전달받아 다익스트라 알고리즘을 수행한다. 함수 내부에서 dist와 visited를 이용하여 최단 거리를 계산하고 탐색이 끝나면 dist를 반환한다.

다익스트라 함수에서는 가장 가까운 노드 를선택하기 위해서 방문 하지 않은 노드 중 현재 dist 값이 가장 작은 노드를 탐색하여 u에 저장한다.

```
int u = -1;
for(int j = 0; j < 5; j++){
    if(!visited[j] && (u == -1 || dist[j] < dist[u])){
        u = j;
    }
}
```

이를 통해 현재까지 발견된 경로 중 시작 노드에서 가장 가까운 미방문 노드를 다음 탐색 대상으로 선택한다. 가장 가까운 노드를 선택한 후 위에서 설명했던 visited 배열을 사용해 해당 노드를 방문한 것으로 처리한다. 방문 처리된 노드는 시작점에서 해당 노드까지의 최단 거리가 확정된 것으로 판단한다. 그리고 현재 노드 u와 연결되어 있는 모든 인접 노드를 확인한다.

```
for(auto& a : graph[u]){
    int v = a.first;
    int weight = a.second;
    if(dist[u] + weight < dist[v]){
        dist[v] = dist[u] + weight;
    }
}
```

현재 노드를 거쳐 인접 노드로 이동하는 비용 dist[u] + weight와 기존에 저장되어 있던 dist[v]를 비교한다. 현재 노드를 거쳐가는 경로가 더 짧은 경우 새로운 거리로 갱신한다. 새로운 거리 = 현재 노드까지의 최단 거리 + 간선의 가중치 dist[u] + weight < dist[v]를 만족할 경우 거리를 갱신한다.

위의 과정을 반복한다. 모든 탐색이 완료되면 dist에는 시작 노드에서 각 노드까지의 최단

거리가 저장되며 이를 반환한다. 전체적인 프로그램의 로직은 다음과 같다.

1. 그래프 생성
2. dist를 INF로 초기화
3. 시작 노드의 dist를 0으로 설정
4. 가장 가까운 미방문 노드 선택
5. 선택한 노드 방문 처리
6. 인접 노드 거리 비교 및 갱신
7. 반복
8. 최종 최단 거리 반환

2-3-(2) A* 알고리즘 C++에서 구현하기

A* 알고리즘을 C++에서 구현해보자.

본 C++ A* 알고리즘 프로그램은 5×5 크기의 미로에서 시작 지점부터 도착 지점까지의 최단 경로를 찾는 프로그램이다. 미로는 0과 1로 구성하였으며 0은 이동 가능한 길, 1은 이동할 수 없는 벽을 의미한다. 시작 위치는 (0, 0), 도착 위치는 (4, 4)로 설정하였다. A* 알고리즘은 시작 지점에서 현재 위치까지의 실제 이동 비용인 g값과 현재 위치에서 도착 지점까지의 예상 거리인 h값을 더한 f값을 이용하여 탐색한다.

먼저 A* 알고리즘에서 각 위치의 정보를 저장하기 위해 Node 구조체를 사용하였다.

```
struct Node {  
    int x, y;  
    int g, h, f;  
};
```

x, y에는 현재 위치의 좌표를 저장한다. g, h는 위에서 설명했던 휴리스틱 함수의 항들이다. 다음으로 f값이 작은 노드를 우선적으로 탐색하기 위해 `priority\_queue`를 사용하였다.

```
priority_queue<Node, vector<Node>, Compare> pq;
```

C++의 priority_queue는 기본적으로 큰 값을 우선적으로 꺼내기 때문에 다음과 같이 Compare 구조체를 만들어 f값이 작은 노드가 먼저 나오도록 설정하였다.

```
struct Compare {  
    bool operator()(const Node& a, const Node& b) { return a.f > b.f; }  
};
```

이를 통해 여러 개의 탐색 후보 중 f값이 가장 작은 위치를 우선적으로 선택할 수 있다.

현재 위치에서 도착 위치까지의 예상 거리 h 를 계산하기 위해 `heuristic()` 함수를 사용하였다.

```
int heuristic(int x, int y, int endX, int endY) {  
    return abs(x - endX) + abs(y - endY); // h = |현재 x - 도착 x| + |현재 y - 도착 y|  
}
```

본 프로그램에서는 상하좌우로만 이동할 수 있기 때문에 맨해튼 거리를 사용하였다
예를 들어 현재 위치가 (1, 2)이고 도착 위치가 (4, 4)라면 h 값은 다음과 같다.

$$h = |1 - 4| + |2 - 4| = 5$$

다음으로 시작 위치에서 각 위치까지의 실제 이동 비용을 저장하기 위해 `dist`를 사용하였다.

```
vector<vector<int>> dist(ROW, vector<int>(COL, INF));
```

처음에는 아직 각 위치까지의 경로를 알 수 없기 때문에 모든 값을 `INF`로 초기화하였다. 시작 위치의 이동 비용은 0이므로 다음과 같이 설정하였다.

```
dist[startX][startY] = 0;
```

탐색 중 현재 위치를 거쳐 이동하는 새로운 비용이 기존의 비용보다 작다면 `dist` 값을 갱신한다.

```
if(newG < dist[nx][ny]) {  
    dist[nx][ny] = newG;  
}
```

각 위치의 방문 여부를 확인하기 위해 `visited` 배열도 사용하였다.

```
vector<vector<bool>> visited(ROW, vector<bool>(COL, false));
```

처음에는 모든 위치를 방문하지 않은 상태이므로 `false`로 초기화한다. `priority_queue`에서 노드를 하나 꺼낸 후 이미 방문한 위치라면 넘어가고 방문하지 않았다면 `true`로 변경한다.

```
if(visited[x][y]) continue;  
visited[x][y] = true;  
최종 경로를 저장하기 위해 parent 배열을 사용하였다.  
vector<vector<pair<int, int>> parent(ROW, vector<pair<int, int>>(COL, {-1, -1}));
```

더 짧은 경로가 발견될 경우 현재 위치를 다음 위치의 부모로 저장한다.

```
parent[nx][ny] = {x, y};
```


탐색이 완료된 후 parent 값을 도착 지점부터 시작 지점까지 역으로 따라가면 실제 최단 경로를 구할 수 있다.

A* 알고리즘은

```
vector<pair<int, int>> AStar(vector<vector<int>>& maze, int startX, int startY, int endX, int endY)
```

이 함수는 미로와 시작 위치, 도착 위치를 전달받아 최단 경로를 계산하고 최종 경로를 반환한다.

먼저 시작 위치의 g값을 0으로 설정하고 heuristic 함수를 사용하여 h값을 계산한다.

```
dist[startX][startY] = 0;
int h = heuristic(startX, startY, endX, endY);
pq.push({startX, startY, 0, h, h});
```

시작 위치에서는 g값이 0이기 때문에 f값은 h값과 같다. 계산된 시작 노드를 priority_queue에 저장한다. 후 priority_queue가 비어 있지 않은 동안 탐색을 반복한다.

```
while(!pq.empty()) {
```

priority_queue에서 f값이 가장 작은 노드를 꺼내 현재 탐색 위치로 설정한다.

```
Node current = pq.top();
pq.pop();
```

```
int x = current.x;
int y = current.y;
```

현재 위치가 이미 방문된 위치라면 탐색하지 않고 다음 노드로 넘어간다. 방문하지 않은 위치라면 방문 처리를 한다. 재 위치가 도착 위치와 동일한지도 확인한다.

```
if(x == endX && y == endY) break;
```

목표 지점에 도착하였다면 더 이상 탐색할 필요가 없으므로 반복문을 종료한다.

현재 위치에서 이동 가능한 주변 위치를 확인하기 위해 상하좌우 방향을 나타내는 배열을 사용하였다.

```
int dx[4] = {-1, 1, 0, 0};  
int dy[4] = {0, 0, -1, 1};
```

이를 이용해 현재 위치를 기준으로 상하좌우의 새로운 좌표를 계산한다.

```
int nx = x + dx[i];  
int ny = y + dy[i];
```

새로운 위치가 미로 범위를 벗어나거나 벽인 경우에는 탐색하지 않는다.

```
if(nx < 0 || nx >= ROW || ny < 0 || ny >= COL) continue;  
if(maze[nx][ny] == 1) continue;
```

이동 가능한 위치라면 현재 위치까지의 비용에 1을 더하여 새로운 g값을 계산한다.

```
int newG = dist[x][y] + 1;
```

본 프로그램에서는 한 칸 이동 비용을 모두 1로 설정하였다.

새롭게 계산한 g값이 기존에 저장된 거리보다 작다면 더 짧은 경로를 발견한 것이므로 dist와 parent를 갱신한다.

```
if(newG < dist[nx][ny]) {  
    dist[nx][ny] = newG;  
    parent[nx][ny] = {x, y};  
}
```

그 후 새로운 위치에서 도착 위치까지의 h값을 계산하고 f값을 구한다.

```
int newH = heuristic(nx, ny, endX, endY);  
int newF = newG + newH;
```

계산된 새로운 노드를 priority_queue에 추가한다.
pq.push({nx, ny, newG, newH, newF});

이 과정을 반복하면서 f값이 가장 작은 위치를 우선적으로 탐색한다. 색이 완료된 후 도착 위치까지 경로가 존재하는지 확인한다.

```

if(dist[endX][endY] == INF) {
    cout << "경로를 찾을 수 없습니다." << endl;
    return {};
}

```

도착 위치의 dist 값이 INF라면 경로를 찾지 못한 것이다.

경로가 존재한다면 `parent` 배열을 이용하여 도착 위치부터 시작 위치까지 역으로 이동하면서 경로를 저장한다.

```

while(true) {
    path.push_back({x, y});

    if(x == startX && y == startY) break;

    pair<int, int> p = parent[x][y];
    x = p.first;
    y = p.second;
}

```

이때 경로는 도착 지점에서 시작 지점 방향으로 저장되기 때문에 reverse() 함수를 이용하여 순서를 반대로 변경한다.

```
reverse(path.begin(), path.end());
```

이후 시작 지점에서 도착 지점까지의 최단 경로를 반환한다
전체적인 프로그램의 로직은 다음과 같다.

1. 5×5 미로 생성
2. 시작 위치와 도착 위치 설정
3. dist를 INF로 초기화
4. 시작 위치의 g값을 0으로 설정
5. 맨해튼 거리를 이용해 h값 계산
6. $f = g + h$ 계산
7. f값이 가장 작은 노드 선택
8. 선택한 노드 방문 처리
9. 상하좌우 위치 탐색
10. 벽과 미로 범위 확인
11. 더 짧은 경로를 발견하면 dist와 parent 갱신
12. 새로운 h값과 f값 계산
13. priority_queue에 새로운 노드 추가

14. 도착 위치를 찾을 때까지 반복
15. parent를 이용하여 최종 경로 생성
16. 최종 경로와 경로 비용 출력